

ML Inference Engine

Lightweight WASM-compiled neural network runtime that runs quantized models in-browser with zero server dependency.

Rust	WASM	ONNX	WebGPU
------	------	------	--------

VERSION	STATUS	LICENSE	BINARY	TARGETS	UPDATED
0.9.2	Open Source	MIT	1.8 MB gzip	wasm32	Sep 2026

1. Overview

The ML Inference Engine executes quantized neural networks entirely inside the browser. The runtime is written in Rust and compiled to WebAssembly; model weights are fetched as static assets and executed against WebGPU where available, falling back to a SIMD WebAssembly kernel path when it is not. There is no inference server, no per-request network round trip, and no user data leaving the device.

This suits workloads where latency, privacy, or cost make server inference awkward: on-device embedding for local search, small-model classification and reranking, speech-to-intent, and image preprocessing. It is explicitly not a training runtime, and it targets models that fit in device memory after quantization — practically, up to roughly two billion parameters at 4-bit on current desktop hardware.

2. Execution pipeline

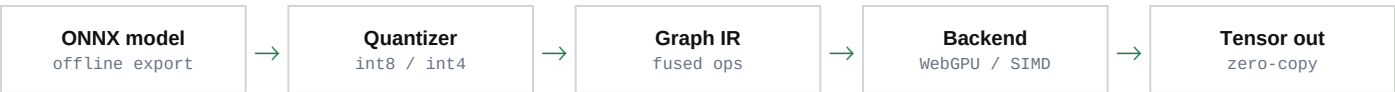


Figure 1 — Ahead-of-time stages (left of the dashed boundary in the repo's build script) run once at publish time; only the last two stages run in the browser.

Conversion is a build-time step. The CLI ingests an ONNX graph, applies per-channel symmetric quantization with a calibration set, folds constants, fuses MatMul + Add + activation chains into single kernels, and emits a compact binary container holding the fused graph plus packed weights. The browser-side loader memory-maps that container into a single WebAssembly linear-memory arena, so model load is one allocation and one fetch rather than thousands of small tensor copies.

3. Runtime layers

JS / TS API	Promise-based surface: <code>load()</code> , <code>run()</code> , <code>stream()</code> . Handles transferables and worker lifecycle.
WASM bridge	Thin wasm-bindgen layer. Passes tensor handles, never copies buffers across the boundary.
Scheduler	Topologically orders the fused graph, allocates from the arena, plans kernel dispatch batches.
Backends	WebGPU compute shaders (WGSL) with a SIMD128 scalar fallback selected at load time by capability probe.
Memory arena	Single pre-sized linear memory region; bump allocation with per-run reset, no GC pressure.

4. Quantization support

SCHEME	WEIGHTS / ACTIVATIONS	TYPICAL QUALITY DELTA	BEST FOR
int8 per-tensor	8-bit / 8-bit	< 0.5% task accuracy	Classifiers, encoders.
int8 per-channel	8-bit / 16-bit	< 0.2% task accuracy	Conv and vision graphs.
int4 grouped	4-bit (g=128) / 16-bit	1–3% perplexity	Decoder-only text models.
fp16 passthrough	16-bit / 16-bit	Reference baseline	WebGPU devices with headroom.

5. Integration

```
import { InferenceEngine } from "@infer/wasm";

const engine = await InferenceEngine.load({
  model:   "/models/intent-classifier.q4.bin",
  backend: "auto",           // "webgpu" | "wasm-simd" | "auto"
  threads: navigator.hardwareConcurrency ?? 4,
  arenaMb: 512,             // fails fast if unavailable
});
// Single-shot inference; input is a transferable Float32Array.
const { logits, timings } = await engine.run({ input_ids: ids });

// Token-streaming for autoregressive graphs.
for await (const tok of engine.stream({ input_ids: ids, maxTokens: 128 })) {
  render(tok.text);
}

engine.dispose();           // releases the arena and GPU device
```

Listing 1 — The engine runs in a dedicated worker by default; the main thread is never blocked.

6. Measured performance

Median of 50 warm runs on an M2 MacBook Air (Chrome 129). Cold start covers fetch from a warm HTTP cache, arena allocation, and shader compilation.

WORKLOAD	WEBGPU	WASM SIMD	COLD START
110M encoder, batch 1, seq 128	9 ms	41 ms	310 ms
1.1B decoder int4, decode step	28 ms/token	190 ms/token	1.4 s
Vision classifier, 224×224	6 ms	22 ms	240 ms
Embedding model, batch 16	17 ms	88 ms	290 ms

7. Constraints and browser support

- WebGPU is required for the fast path: Chrome and Edge 113+, Safari 18+, Firefox 141+. Everything else transparently takes the SIMD fallback, which needs SharedArrayBuffer and therefore cross-origin isolation headers.
- Memory is the binding constraint, not compute. iOS Safari caps WebAssembly memory well below desktop limits; the loader probes the ceiling and refuses a model it cannot hold rather than crashing the tab mid-run.
- Weights are public once shipped to a browser, so models under a restrictive license should stay server-side. Operator coverage is a deliberate subset of ONNX opset 17; unsupported operators fail at conversion time with a named node, never silently at runtime.

8. Roadmap

RELEASE	PLANNED WORK	TARGET
0.10	Paged KV cache with configurable eviction for long-context decoders.	Q4 2026
0.11	Speculative decoding using a distilled draft model in a second worker.	Q1 2027
1.0	Stable binary container format, versioned opset guarantee, LTS branch.	Q2 2027